

Tài liệu học viên — Local LLM · RAG

Xây dựng trợ lý AI truy hồi tài liệu, vận hành hoàn toàn trên máy cục bộ

Workshop Local LLM · RAG

Tháng 6 / 2026

Mục lục

Phần I — Sử dụng Open WebUI cho RAG (không cần code)

- Tóm tắt quy trình
- 1. Phạm vi sử dụng tài liệu
- 2. Lý thuyết RAG
- 3. Cài Ollama và chạy LLM cục bộ
- 4. Dùng Open WebUI cho RAG
- 5. Xử lý lỗi thường gặp (thiếu chương trình / không chạy)

Phần II — Tự xây ứng dụng RAG bằng code (nâng cao)

- 1. Mục tiêu xây dựng
- 2. Chuẩn bị
- 3. Cấu hình
- 4. Xây dựng từng bước — 6 bước RAG
- 5. Chạy thử
- 6. Tùy biến theo nhu cầu
- 7. Bổ sung giao diện web (tùy chọn)
- 8. Lỗi thường gặp

Phụ lục — Bảng lệnh cài đặt nhanh (Ollama + Open WebUI)

- A. WINDOWS — mở PowerShell (chuột phải nút Start → *Windows PowerShell*)
- B. macOS — mở Terminal
- C. Trường hợp không sử dụng được Docker: cài bằng pip (cả 2 OS, cần Python 3.11+)
- D. Lần đầu vào Open WebUI (BẮT BUỘC cho RAG)
- E. Xử lý lỗi thường gặp
- F. Lệnh kiểm tra / quản lý nhanh

Phần I — Sử dụng Open WebUI cho RAG (không cần code)

Tài liệu này được sử dụng khi tiến độ thực hành trên lớp không theo kịp, hoặc khi một thuật ngữ hay một bước thao tác chưa rõ. Không bắt buộc đọc tuần tự; tra cứu đúng mục cần thiết. Nội dung khớp với buổi workshop: xây dựng trợ lý RAG chạy hoàn toàn trên máy cục bộ thông qua Open WebUI (không cần code). Quan trọng: Dữ liệu không rời khỏi máy. Cả LLM lẫn mô hình nhúng đều chạy cục bộ qua Ollama; tài liệu nạp vào Open WebUI không được gửi lên đám mây. Đây là lý do sử dụng RAG cục bộ thay vì dán nội dung vào ChatGPT.

Tóm tắt quy trình

Năm bước dưới đây là quy trình tối thiểu để có một trợ lý RAG hoạt động (mỗi bước trỏ tới mục chi tiết tương ứng).

1. Cài và chạy Ollama, tải hai mô hình: `ollama pull qwen3:1.7b` và `ollama pull nomic-embed-text` (mục 3).
2. Khởi động Open WebUI: trong thư mục repo, thực hiện `docker compose up -d`, sau đó mở `http://localhost:3000` (trường hợp cài bằng pip thì `http://localhost:8080`) (mục 4.1).
3. Đổi mô hình nhúng sang chạy cục bộ — bắt buộc và phải thực hiện trước khi nạp tài liệu: Admin Panel → Settings → Documents → Engine = **Ollama**, Model = **nomic-embed-text** → Save (mục 4.4).
4. Nạp tài liệu: tạo Knowledge base rồi upload (bộ mẫu có sẵn trong `2_rag/sample_upload/`) (mục 4.3).
5. Truy vấn có trích nguồn: gõ # để chọn kho, sau đó đặt câu hỏi đầy đủ. Sự xuất hiện của mục **Sources** dưới câu trả lời xác nhận RAG đã hoạt động (mục 4.6).

Lưu ý: Nếu câu trả lời rất chậm, nguyên nhân thường là qwen3 đang ở chế độ “suy nghĩ”; thêm `/no_think` vào đầu câu hỏi hoặc tắt nút “Think” (mục 5.8).

1. Phạm vi sử dụng tài liệu

Tài liệu này dùng để đọc thêm khi tiến độ thực hành trên lớp không theo kịp, hoặc khi gặp một thuật ngữ chưa hiểu. Đây không phải tài liệu cần đọc tuần tự từ đầu đến cuối; nên tra cứu đúng mục đang cần. Khi gặp một thuật ngữ lạ (ví dụ “embedding”, “chunk”, “top-k”), tra Bảng thuật ngữ bên dưới; khi gặp một bước thao tác, xem phần hướng dẫn từng bước tương ứng. Thời gian đọc lướt toàn bộ khoảng 15-20 phút.

Bảng thuật ngữ

Thuật ngữ	Nghĩa (dễ hiểu)
LLM (Large Language Model)	Mô hình ngôn ngữ lớn, được huấn luyện để

Thuật ngữ	Nghĩa (dễ hiểu)
	hiểu và sinh văn bản; là “bộ não” trả lời câu hỏi.
Token	Đơn vị văn bản nhỏ mà mô hình xử lý (một từ thường tách thành 1-vài token); chi phí và độ dài đều tính theo token.
Context window	Lượng token tối đa mô hình “nhìn thấy” cùng lúc (câu hỏi + tài liệu + lịch sử); vượt quá sẽ bị cắt bớt.
Local / Offline	Chạy mô hình ngay trên máy cục bộ, không gửi dữ liệu lên mạng — riêng tư và không cần Internet.
Quantization	Kỹ thuật nén mô hình (giảm độ chính xác số) để chạy nhẹ hơn, tốn ít RAM hơn, đổi lại chất lượng giảm nhẹ.
Ollama	Phần mềm chạy LLM trên máy cá nhân; tải, quản lý mô hình và cung cấp API tại cổng 11434.
Docker	Công cụ đóng gói phần mềm kèm mọi thứ nó cần, để chạy giống nhau trên mọi máy.
Container	Một “hộp” cô lập do Docker tạo ra, chứa sẵn ứng dụng đang chạy (ví dụ Open WebUI).
Open WebUI	Giao diện web để trò chuyện với LLM và quản lý tài liệu, thao tác bằng chuột — không cần code.
RAG (Retrieval-Augmented Generation)	Kỹ thuật cho LLM truy hồi tài liệu trước rồi mới trả lời, giúp chính xác và giảm bịa.
Embedding	Cách biến một đoạn văn bản thành dãy số (vector) thể hiện ý nghĩa, để máy so sánh độ tương đồng.
Vector	Dãy số biểu diễn ý nghĩa của văn bản; hai văn bản gần nghĩa thì hai vector gần nhau.
Vector DB (ChromaDB)	Cơ sở dữ liệu lưu các vector và tìm nhanh những đoạn gần nghĩa nhất với câu hỏi.
Chunk / Chunking	Việc cắt tài liệu dài thành các đoạn nhỏ để nhúng và truy hồi chính xác hơn.
Overlap	Phần văn bản lặp giữa hai chunk liền kề, giúp không mất ngữ cảnh ở ranh giới cắt.
Top-k	Số đoạn gần nghĩa nhất được lấy ra để đưa cho LLM tham khảo (ví dụ top-k = 3 lấy 3

Thuật ngữ	Nghĩa (dễ hiểu)
	đoạn).
Knowledge base	Bộ tài liệu được nạp vào hệ thống làm nguồn truy hồi cho RAG.
System Prompt	Câu chỉ dẫn nền đặt sẵn cho mô hình, quy định vai trò và cách trả lời trong suốt cuộc trò chuyện.

2. Lý thuyết RAG

RAG là gì và vì sao cần

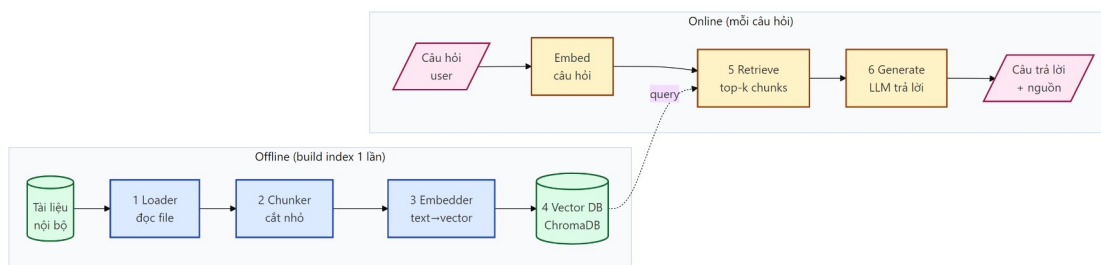
LLM được huấn luyện trên dữ liệu công khai — nó không biết tài liệu riêng (quy chế nội bộ, sổ tay, hợp đồng). Khi hỏi trực tiếp, mô hình có thể bịa trơn tru (hallucination). Còn nếu dán cả tập tài liệu vào câu hỏi thì vượt context window, tốn token và chậm.

RAG (Retrieval-Augmented Generation) giải quyết vấn đề bằng cách chỉ truy hồi đúng vài đoạn liên quan nhất rồi đưa cho LLM trả lời. Theo ẩn dụ: hỏi trực tiếp tương tự thi đóng sách (nhớ gì nói nấy); RAG tương tự thi mở sách — tra đúng trang rồi mới trả lời.

Pipeline 6 bước

Quy trình gồm 4 bước OFFLINE (thực hiện một lần khi dựng kho) và 2 bước ONLINE (lặp lại ở mỗi câu hỏi):

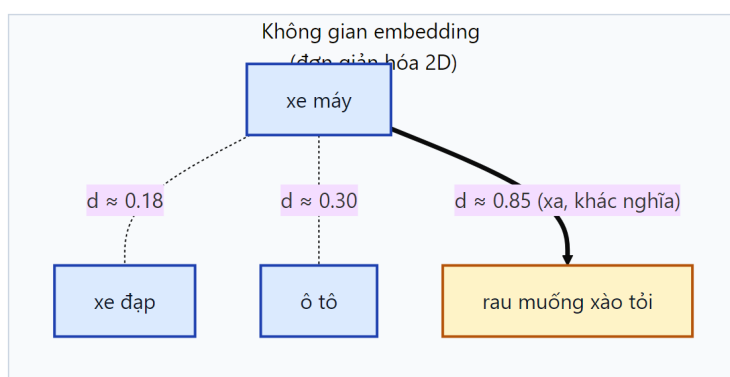
Pha	Bước	Việc làm
OFFLINE	1. Load	Nạp tài liệu
OFFLINE	2. Chunk	Cắt thành đoạn nhỏ
OFFLINE	3. Embed	Biến mỗi đoạn thành vector mang nghĩa
OFFLINE	4. Store	Lưu vector vào vector DB
ONLINE	5. Retrieve	Truy hồi các đoạn gần nghĩa nhất với câu hỏi
ONLINE	6. Generate	Đưa các đoạn cho LLM sinh câu trả lời



Embedding

Embedding biến một đoạn văn bản thành vector — dãy số mô tả ý nghĩa (nomic-embed-text cho vector 768 chiều; mô hình khác có thể nhiều hoặc ít hơn). Nguyên tắc cốt lõi: nghĩa gần nhau thì vector gần nhau trong không gian, đo bằng độ tương đồng cosine.

Cách này vượt trội so với tìm từ khóa: “mã đăng nhập” và “mật khẩu” khác hẳn về chữ nên tìm từ khóa sẽ trượt, nhưng gần nhau về nghĩa nên embedding vẫn khớp. Buổi học dùng mô hình nomic-embed-text, chạy hoàn toàn cục bộ.



Chunking, Vector DB và Top-k

- Chunking: cắt tài liệu thành đoạn khoảng 500 ký tự, overlap khoảng 50 ký tự để không đứt ý ở ranh giới đoạn.
- Vector DB (ChromaDB): mỗi đoạn lưu đồng thời ba thứ — vector, văn bản gốc và metadata (ví dụ tên file); dùng thuật toán HNSW để tìm hàng xóm gần nhất rất nhanh dù có nhiều đoạn.
- Top-k: mỗi câu hỏi chỉ lấy k đoạn gần nghĩa nhất đưa cho LLM. Buổi học đặt $k = 3$.

Giới hạn và Đánh giá

RAG không phải phép màu — nếu bước Retrieve truy hồi nhầm đoạn, LLM sẽ trả lời sai theo (“rác vào, rác ra”). Câu hỏi nằm ngoài tài liệu được chặn bằng System Prompt (yêu cầu chỉ dùng tài liệu, thiếu thì đáp “không đề cập”).

Tiêu chí đánh giá quan trọng nhất là Faithfulness — câu trả lời bám sát nguồn, có trích dẫn, không thêm thông tin ngoài nguồn. Việc đầu tiên khi nhận một câu trả lời là mở Sources để xem nguồn. Đánh giá bài bản có thể dùng khung RAGAS để chấm điểm tự động.

3. Cài Ollama và chạy LLM cục bộ

Ollama là công cụ tải và chạy mô hình AI (LLM) ngay trên máy cục bộ. Chỉ cần vài lệnh, không cần biết code. Phần này thực hiện một lần duy nhất.

Tự động hoá: Repo có sẵn script thực hiện toàn bộ các bước dưới đây (setup.ps1 và pull_models.ps1). Cách chạy: mở PowerShell, cd vào thư mục repo, thực hiện .\0_setup\setup.ps1 (nếu báo “*running scripts is disabled*” thì xem mục 5.1). Phần này giải thích từng lệnh thủ công để hiểu rõ khi script báo lỗi. Bảng lệnh gọn để copy-paste cho máy mới (Win/Mac) có trong **Phụ lục**.

3.1 Cài Ollama

Yêu cầu: RAM tối thiểu 8GB, ổ trống còn ≥ 5 GB.

Mở terminal (Windows: chuột phải nút Start → *Windows PowerShell*; macOS/Linux: mở app Terminal), rồi nhập lệnh theo đúng hệ điều hành đang dùng.

Lưu ý về Terminal trên Windows: nhấn **Start**, gõ “PowerShell”, mở **Windows PowerShell** (cửa sổ nền xanh hoặc đen). Đây là nơi nhập lệnh rồi nhấn Enter. Chỉ nhập phần lệnh, không nhập ký tự nhắc đầu dòng (> hay \$). Trên macOS: nhấn **Cmd+Space**, gõ “Terminal”, nhấn Enter.

Hệ điều hành	Lệnh cài
Windows	winget install Ollama.Ollama
macOS / Linux	curl -fsSL https://ollama.com/install.sh sh

Cài xong, kiểm tra kết quả:

```
ollama --version
```

Nếu in ra một dòng số phiên bản (ví dụ ollama version 0.x.x) là đã cài xong.

Trường hợp Windows báo winget không nhận diện: vào Microsoft Store cài *App Installer*, hoặc tải bản cài thủ công trực tiếp tại **ollama.com/download** rồi cài như phần mềm thông thường.

Trường hợp sau khi cài lần đầu trên Windows mà gõ ollama báo lệnh không tồn tại: đóng cửa sổ PowerShell hiện tại (gõ exit), mở cửa sổ PowerShell MỚI, rồi thử lại. Nguyên nhân là máy cần nạp lại biến môi trường.

3.2 Lấy mô hình (tải mô hình về máy)

Chọn mô hình theo dung lượng RAM của máy. Chọn mô hình quá nặng so với cấu hình sẽ khiến hệ thống chạy rất chậm hoặc treo.

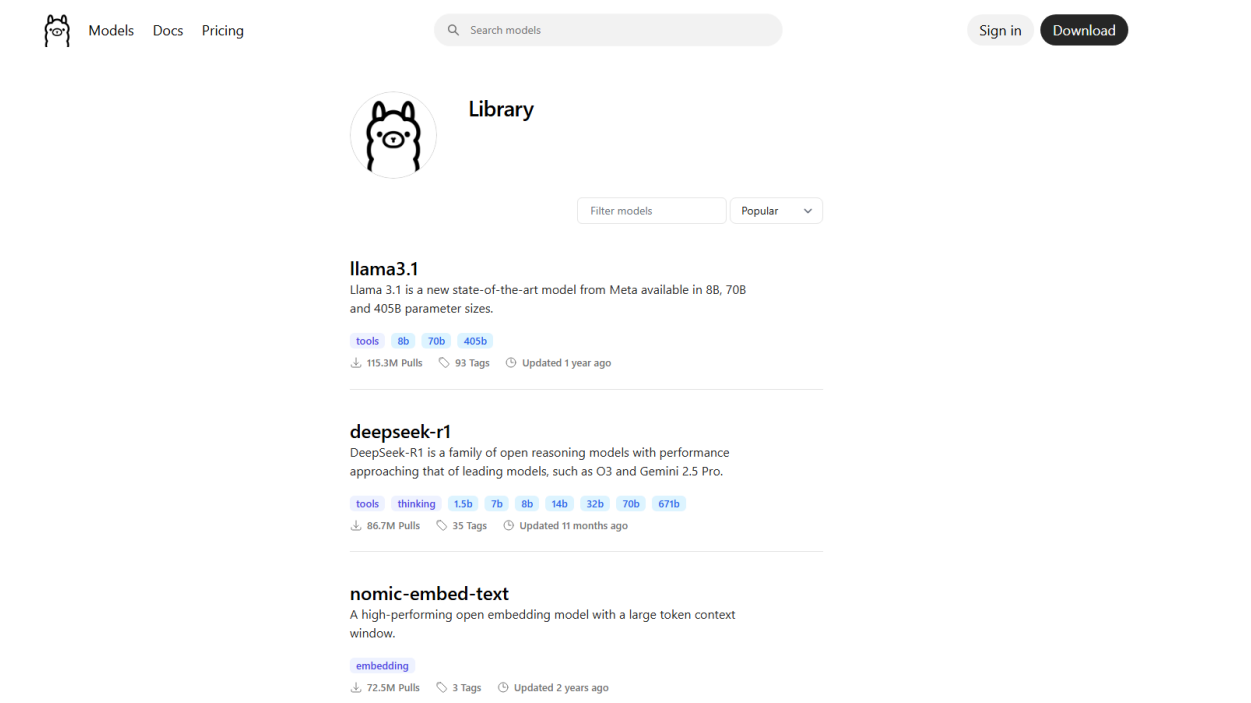
RAM / Phần cứng	Model LLM	Dung lượng	Ghi chú
8GB (hoặc không có GPU)	qwen3:1.7b	~1.4GB	Mặc định — dùng xuyên suốt workshop
16GB	qwen3:4b	~2.5GB	Trả lời chất lượng hơn
Có GPU rồi	qwen3:8b	~5GB	Tốt nhất, cần máy mạnh

Tải mô hình LLM và mô hình nhúng (nomic-embed-text dùng cho RAG ở Module 2) bằng hai lệnh:

```
ollama pull qwen3:1.7b
ollama pull nomic-embed-text
```

Lần đầu tải mất vài phút tùy tốc độ mạng. Trong lúc tải, thanh % chạy dần; quá trình hoàn tất sẽ hiện success. Kiểm tra bằng `ollama list` — phải thấy cả `qwen3:1.7b` và `nomic-embed-text`. Sau đó mô hình chạy hoàn toàn offline, không cần Internet nữa.

Danh sách mô hình có sẵn được liệt kê tại trang thư viện chính thức ollama.com/library:



Chi tiết mô hình qwen3 có thể xem ngay trong thư viện đó:



qwen3

↓ 30M Downloads ⌚ Updated 7 months ago

Qwen3 is the latest generation of large language models in Qwen series, offering a comprehensive suite of dense and mixture-of-experts (MoE) models.

[tools](#) [thinking](#) [0.6b](#) [1.7b](#) [4b](#) [8b](#) [14b](#) [30b](#) [32b](#) [235b](#)

CLI [cURL](#) [Python](#) [JavaScript](#)

```
ollama run qwen3
```

Applications



Claude Code

```
ollama launch claude --model qwen3
```



Codex App

```
ollama launch codex-app --model qwen3
```



OpenClaw

```
ollama launch openclaw --model qwen3
```



Hermes Agent

```
ollama launch hermes --model qwen3
```



Codex

```
ollama launch codex --model qwen3
```



OpenCode

```
ollama launch opencode --model qwen3
```

3.3 Chạy thử

Cách 1 — Hỏi nhanh một câu (chạy xong tự thoát):

```
ollama run qwen3:1.7b "Giải thích RAG trong 3 câu"
```

Cách 2 — Mở phiên chat liên tục (nhập nhiều câu hỏi). Sau khi nhập lệnh, dấu nhắc đổi thành >>>; nhập câu hỏi rồi nhấn Enter. Để thoát, gõ /bye:

```
ollama run qwen3:1.7b
```

Độ trễ ở truy vấn đầu tiên: Lần đầu hỏi, hệ thống có thể im lặng 15–25 giây trước khi chữ xuất hiện. Đây là lúc Ollama nạp mô hình vào bộ nhớ (đặc biệt khi chạy bằng CPU, không có GPU). Sau đó chữ sẽ xuất hiện dần.

Kết quả mong đợi như sau — mô hình trả lời ngay trong terminal:

```
Windows PowerShell - ollama run qwen3:1.7b

PS C:\Users\Minh Phuc\...\bai-giang> ollama run qwen3:1.7b "Giải thích RAG trong 3 câu"

RAG (Retrieval-Augmented Generation) cho LLM đọc tài liệu của bạn trước khi trả lời: hệ thống tìm các đoạn liên quan nhất rồi đưa kèm câu hỏi vào model. Nhờ vậy câu trả lời bám tài liệu, ít bịa, và trích được nguồn để kiểm chứng.

PS C:\Users\Minh Phuc\...\bai-giang>

[100% offline] qwen3:1.7b (~1.4GB) chạy ngay trên máy · không gọi cloud
```

Khi mô hình trả lời được, một mô hình AI đã chạy thành công ngay trên máy cục bộ. Bước tiếp theo là chuyển sang giao diện Open WebUI để thao tác thuận tiện hơn.

Lưu ý: Ollama cần tiếp tục chạy nền (không Quit) vì Open WebUI ở phần 4 cần nó để trả lời. Trên Windows, Ollama nằm ở khay hệ thống góc phải.

4. Dùng Open WebUI cho RAG

Open WebUI là giao diện chat tương tự ChatGPT nhưng chạy hoàn toàn trên máy cục bộ. Nó tích hợp sẵn toàn bộ pipeline RAG (lưu vector, cắt đoạn, truy hồi, trích nguồn), do đó RAG được thực hiện hoàn toàn bằng giao diện, không cần viết code. Phần này hướng dẫn từng bước.

4.1 Cài Docker và Open WebUI

Open WebUI chạy gọn nhất qua Docker. Trường hợp máy không cài được Docker (ví dụ máy công ty chặn), phương án thay thế bằng pip được trình bày ở cuối mục này.

Bước 1 — Cài Docker Desktop:

- **Windows:** mở PowerShell, chạy:

```
winget install Docker.DockerDesktop
```

- **macOS:** cần có Homebrew trước (xem brew.sh), rồi chạy:

```
brew install --cask docker
```

Bước 2 — Mở Docker Desktop và đợi đến khi trạng thái hiện “running” (biểu tượng cá voi ở góc màn hình hết nhấp nháy). Đây là bước hay bị bỏ quên; nếu Docker chưa “running”, lệnh ở bước 3 sẽ báo lỗi.

Bước 3 — Khởi động Open WebUI. Trước tiên đưa PowerShell vào đúng thư mục repo (nơi có file docker-compose.yml): nhập cd kèm đường dẫn, ví dụ cd "C:\Users\TenBan\Downloads\bai-giang-localLLM-RAG-Agent" (có thể copy đường dẫn ở thanh địa chỉ File Explorer rồi dán sau cd). Nhập dir để kiểm tra — phải thấy docker-compose.yml trong danh sách. Sau đó chạy:

```
docker compose up -d
```

Lệnh trả về dấu nhắc ngay, nhưng lần đầu cần thêm 1–3 phút để tải image và khởi động. Để kiểm tra hệ thống đã sẵn sàng, nhập docker ps — phải thấy dòng open-webui ở trạng thái **Up**. Khi đó, mở trình duyệt vào:

```
http://localhost:3000
```

Cấu hình mẫu của workshop tắt đăng nhập (WEBUI_AUTH=False) nên giao diện được truy cập trực tiếp, không cần tạo tài khoản. Khi triển khai thật cho nhiều người, bật lại để phân quyền.

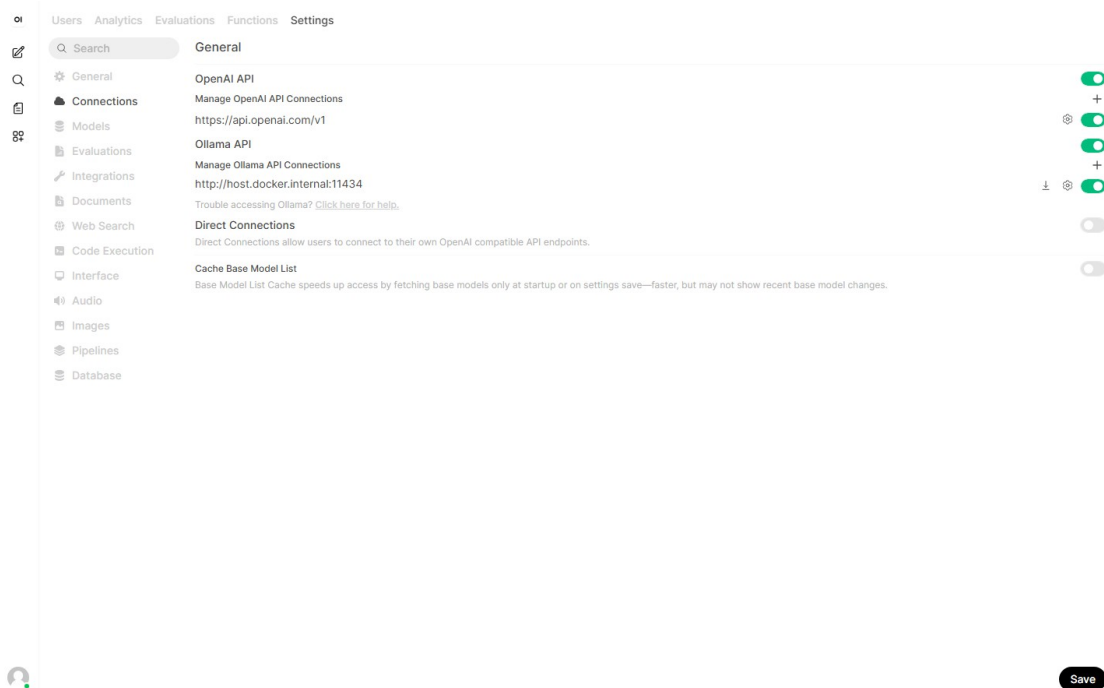
Trường hợp không dùng được Docker, cài trực tiếp bằng pip:

```
pip install open-webui
open-webui serve
```

Sau đó mở địa chỉ mà terminal in ra (mặc định `http://localhost:8080`). Lưu ý: lần đầu cài rất nặng (~2.5GB) và mất khá nhiều thời gian. Các bước 4.2–4.6 phía dưới thực hiện y hệt, chỉ khác địa chỉ (:8080 thay vì :3000).

4.2 Mở và kiểm tra kết nối Ollama

1. Mở `http://localhost:3000`.
2. Ở khung chọn mô hình phía trên, chọn **qwen3:1.7b**. (Nếu khung trống, không có mô hình, Open WebUI chưa kết nối được Ollama: kiểm tra app Ollama đã chạy — mở `http://localhost:11434` phải thấy “Ollama is running” — rồi tải lại trang F5; xem mục 5.2.)
3. Kiểm tra Open WebUI đã “nhìn thấy” Ollama chưa: bấm góc dưới trái (tên người dùng) → **Admin Panel** → **Settings** → **Connections**. Mục **Ollama API** phải là `http://host.docker.internal:11434` và công tắc đang bật xanh.



Vì sao là `host.docker.internal` chứ không phải `localhost`: Open WebUI chạy trong Docker (một “máy ảo nhỏ”), nên với nó `localhost` là chính nó, không phải máy thật. `host.docker.internal` là cách Docker gọi ra “máy thật bên ngoài” — nơi Ollama đang chạy. Trường hợp cài bằng pip thì điền `http://localhost:11434`.

4.3 Nạp tài liệu vào (2 cách)

Quan trọng: Thực hiện mục 4.4 trước (đổi Embedding sang `nomic-embed-text`) rồi mới nạp tài liệu. Nếu nạp trước, tài liệu bị nhúng bằng embedding mặc định (sai) và phải làm lại (Reindex hoặc xóa rồi nạp lại).

Trường hợp chưa có tài liệu để thử: Repo có sẵn bộ mẫu (công ty giả định “ABC”): `2_rag/sample_upload/` (6 file, 6 định dạng pdf/docx/txt/csv/xlsx/md) và

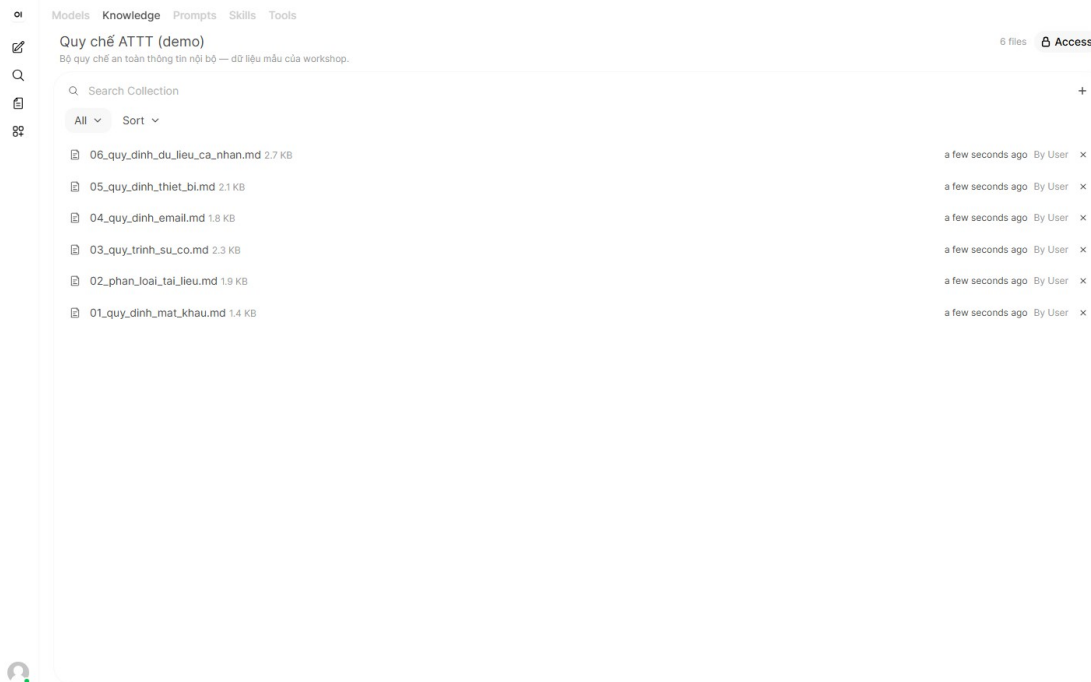
2_rag/data/ (6 file .md quy chế ATTT). Upload thư mục 2_rag/data/ để khớp các câu hỏi mẫu ở mục 4.6.

Cách (a) — NHANH, dùng cho một cuộc chat: 1. Trong ô chat, bấm nút + (góc trái dưới) hoặc kéo–thả thẳng một file vào ô chat. 2. Đợi file xử lý xong, rồi hỏi bình thường. Cách này tiện để thử nhanh, nhưng file không được lưu lại cho lần sau.

Cách (b) — BỀN, tạo Knowledge base (kho tài liệu) để dùng lại nhiều lần: 1. Sidebar trái → **Workspace** → **Knowledge** → bấm + **New Knowledge**. 2. Đặt tên kho (ví dụ “Quy chế ATTT”) → bấm **Create**.

The screenshot shows the 'Create a knowledge base' interface. At the top, there's a navigation bar with 'Models', 'Knowledge', 'Prompts', 'Skills', and 'Tools'. Below it, a sidebar on the left contains icons for chat, search, and other functions. The main area is titled 'Create a knowledge base' and has two input fields: 'What are you working on?' with the text 'Quy chế ATTT (demo)' and 'What are you trying to achieve?' with the text 'Bộ quy chế an toàn thông tin nội bộ — dữ liệu mẫu của workshop.' Below these fields, there's a privacy setting section with a lock icon, the word 'Private', and a dropdown arrow. A note says 'Only select users and groups with permission can access'. Underneath is an 'Access List' section with a '+ Add Access' button and a message 'No access grants. Private to you.' At the bottom right, there is a 'Create Knowledge' button.

3. Mở kho vừa tạo → bấm + → upload nhiều file cùng lúc. Hỗ trợ nhiều định dạng: .md, .txt, .pdf, .docx, .csv, .xlsx.

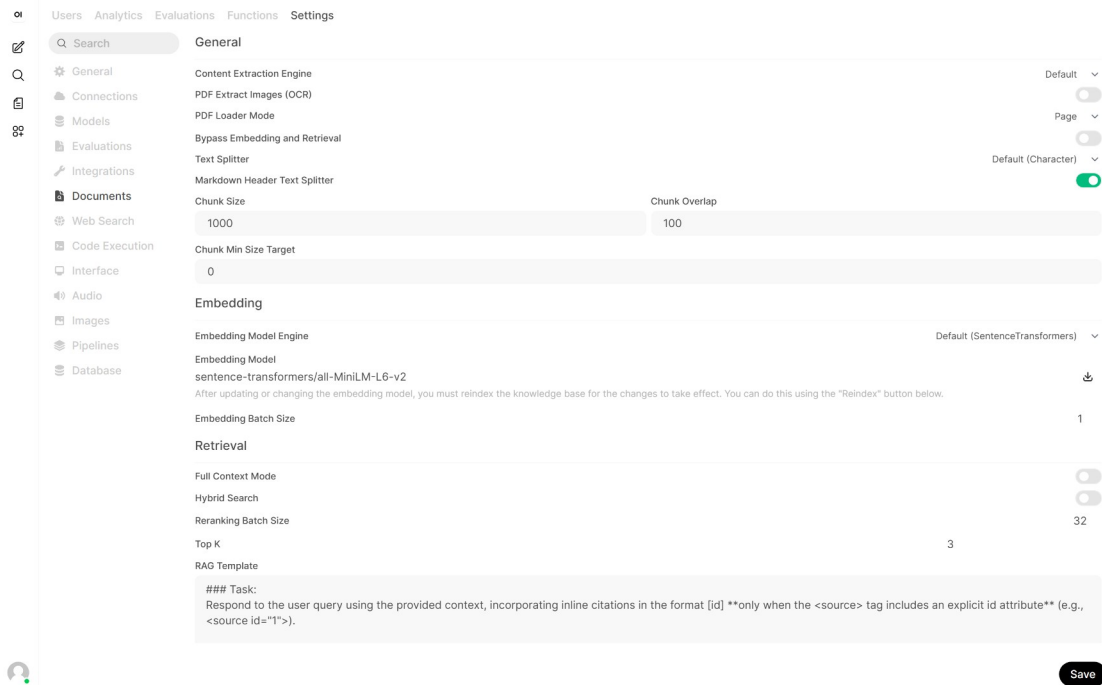


4. Khi chat, gõ ký tự # — một danh sách hiện ra — chọn đúng kho tài liệu rồi đặt câu hỏi.



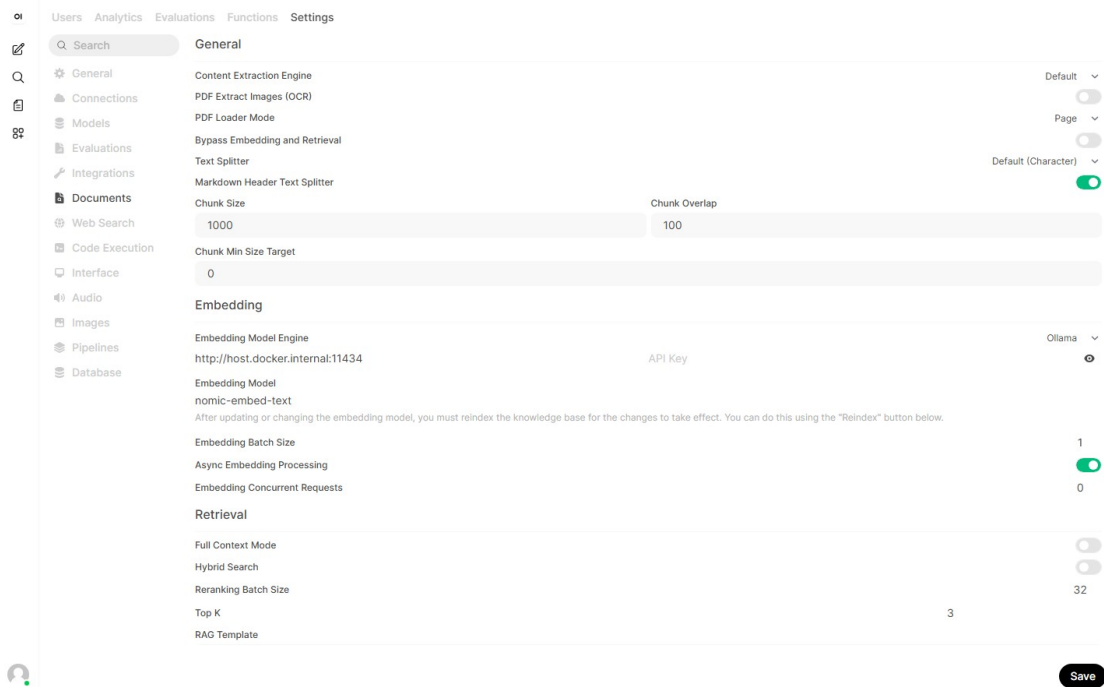
4.4 Chỉnh thông số RAG (quan trọng — phải làm trước khi nạp tài liệu)

Vào **Admin Panel** → **Settings** → **Documents**.



Bắt buộc đổi embedding sang chạy cục bộ. Mặc định Open WebUI dùng sentence-transformers (all-MiniLM) — không phải embedding chạy qua Ollama thuần như cấu hình của workshop. Cần đổi: - **Embedding Model Engine = Ollama - Embedding Model = nomic-embed-text**

Sau khi đổi, bấm **Save**. Tài liệu đã nạp trước đó cần được nhúng lại bằng mô hình mới — dùng nút **Reset/Reindex** trong Settings → Documents nếu có, hoặc xóa và upload lại. Khuyến nghị đổi Embedding trước khi nạp tài liệu — xem cảnh báo ở mục 4.3.



Các thông số nên đặt (giá trị khuyến nghị cho workshop):

Thông số	Giá trị	Ý nghĩa (dễ hiểu)
Embedding Model Engine	Ollama	Dùng đúng embedding cục bộ
Embedding Model	nomic-embed-text	Mô hình biến chữ thành vector, chạy trên máy cục bộ
Chunk Size	500	Mỗi đoạn tài liệu dài khoảng 500 ký tự
Chunk Overlap	50	Hai đoạn liền kề gối nhau 50 ký tự (đỡ đứt ý)
Top K	3	Mỗi câu hỏi lấy 3 đoạn liên quan nhất để trả lời
Hybrid Search	Bật nếu tài liệu có mã/từ khóa hiếm	Kết hợp tìm theo nghĩa và tìm theo từ khóa

Đổi xong, bấm **Save**.

4.5 Tạo trợ lý RAG chuyên dụng (không phải gõ # mỗi lần)

Phân biệt: *Knowledge base* (kho tài liệu) là nơi chứa file (tạo ở 4.3). *Model / Trợ lý* là thành phần trả lời, đã gắn sẵn kho tài liệu đó kèm chỉ dẫn cách trả lời (tạo ở 4.5).

Thay vì mỗi lần chat phải gõ # để chọn kho, có thể tạo sẵn một “trợ lý” đã gắn kho tài liệu — chọn một lần là hỏi nhiều lần.

1. **Workspace** → **Models** → **New Model**.
2. **Base model**: chọn **qwen3:1.7b**.
3. **System Prompt** (chỉ dẫn cố định cho trợ lý), ví dụ:

/no_think

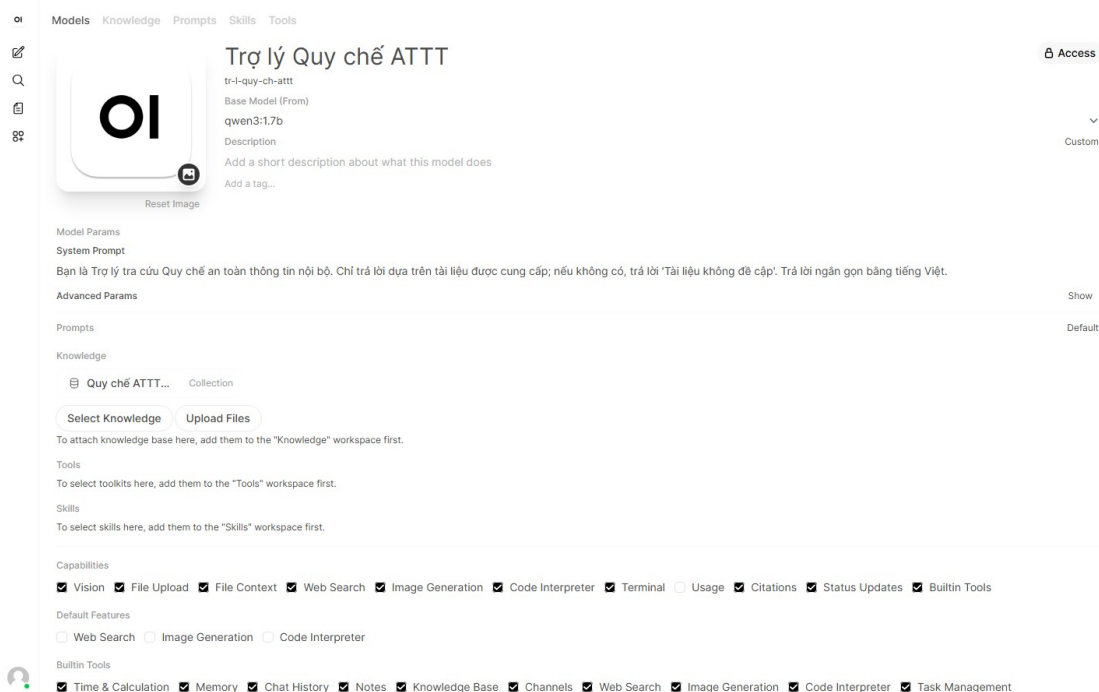
Bạn là trợ lý chỉ trả lời dựa trên tài liệu được cung cấp.

Nếu thông tin không có trong tài liệu, hãy trả lời: "Tài liệu không đề cập."

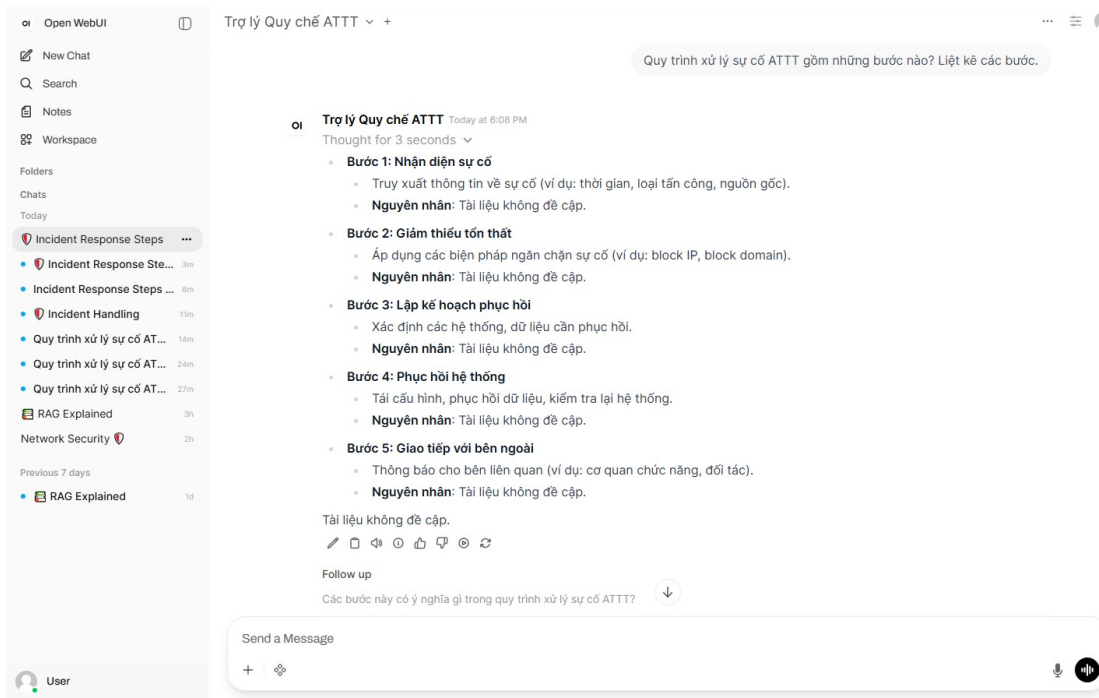
Luôn trả lời bằng tiếng Việt.

Dòng đầu /no_think tắt chế độ “suy nghĩ” của qwen3. Không có nó, mô hình sinh một đoạn lý luận dài (thường bằng tiếng Anh) trước khi trả lời, làm câu trả lời rất chậm.

4. Kéo xuống mục **Knowledge** → bấm **Select Knowledge** → chọn kho tài liệu đã tạo ở 4.3.
5. Bấm **Save & Create**.



Sau đó, quay lại ô chat, ở khung chọn mô hình chọn trợ lý vừa tạo; từ đó hỏi bình thường, không cần gõ # nữa.



4.6 Hỏi đáp có trích nguồn

Lưu ý quan trọng: đặt câu hỏi bằng câu tự nhiên, đầy đủ. Tránh nhập mã ngắn hay viết tắt kiểu "P1", "MFA", vì embedding so khớp theo nghĩa, mã quá ngắn dễ làm truy hồi bị trượt (lấy nhầm đoạn).

Độ trễ ở truy vấn đầu tiên: Khi hỏi lần đầu, màn hình có thể đứng yên vài chục giây do qwen3 đang ở chế độ “suy nghĩ”. Để trả lời ngay: tắt nút **“Think”** (icon bộ não) cạnh ô gửi, hoặc đặt /no_think ở trợ lý (mục 4.5). Xem mục 5.8.

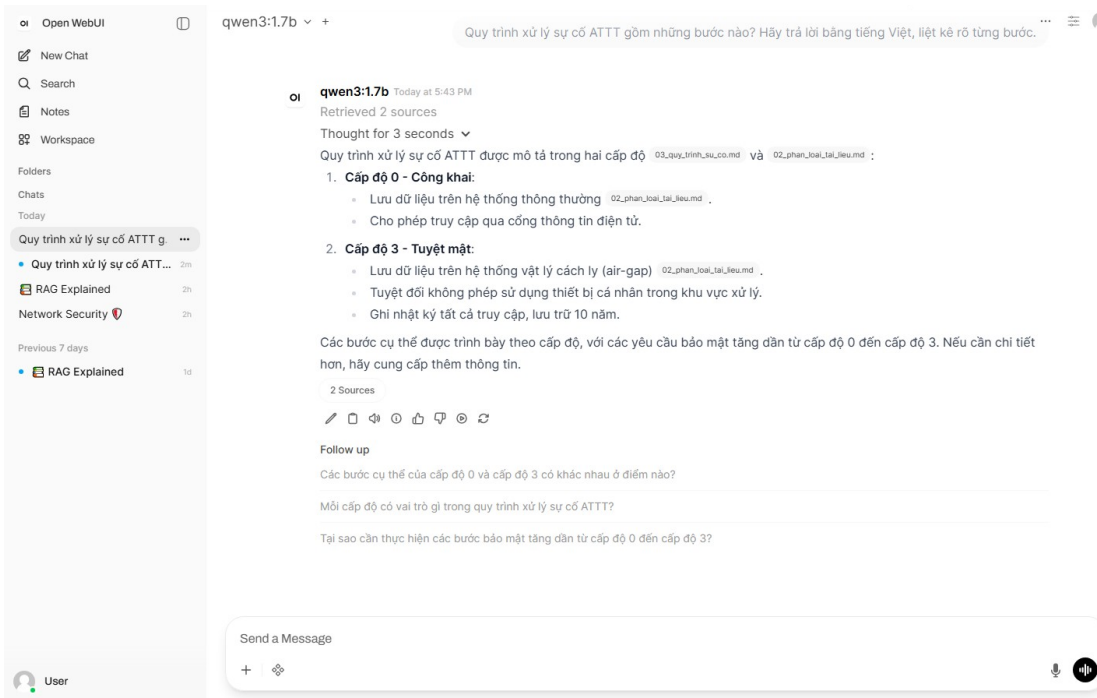
Câu hỏi mẫu chạy ổn định trên bộ tài liệu mẫu (2_rag/data/):

Quy trình xử lý sự cố ATTT gồm những bước nào?
Mật khẩu phải dài tối thiểu bao nhiêu ký tự và bao lâu phải đổi?
Sự cố mức P1 phải phản hồi trong bao lâu?
Nhân viên có được dán dữ liệu nội bộ vào ChatGPT không?

Câu thử “lá chắn” (đáp án không có trong tài liệu, trợ lý phải đáp “Tài liệu không đề cập”):

Giá cổ phiếu của công ty hôm nay là bao nhiêu?

Trong câu trả lời, dưới phần nội dung có mục **Sources / trích nguồn**. Bấm vào đó để xem đúng đoạn gốc trong tài liệu mà mô hình đã dựa vào; đây là bằng chứng RAG đang hoạt động (mô hình trả lời theo tài liệu, không bịa).



Trường hợp không thấy phần Sources, nguyên nhân thường là: (1) chưa chọn kho bằng # hoặc chưa dùng đúng trợ lý ở 4.5; (2) chưa đổi embedding sang nomic-embed-text rồi Reindex (mục 4.4); (3) câu hỏi quá ngắn hoặc dùng mã viết tắt — thử hỏi lại bằng câu đầy đủ.

Cách xác nhận RAG đã chạy đúng: đủ ba dấu hiệu — (1) dưới câu trả lời có mục **Sources** trở đúng tên file đã nạp; (2) bấm vào Source thấy đúng đoạn gốc trong tài liệu; (3) hỏi câu ngoài tài liệu thì trợ lý đáp **“Tài liệu không đề cập”** (không bịa). Thiếu mục Sources nghĩa là đang chat thuần, chưa phải RAG.

5. Xử lý lỗi thường gặp (thiếu chương trình / không chạy)

Phần này dùng để tra cứu khi hệ thống không chạy. Tìm dòng có triệu chứng trùng với hiện tượng quan sát được, rồi thực hiện theo cột xử lý. Các lệnh dưới đây chạy trong **PowerShell** (Windows) hoặc **Terminal** (macOS/Linux); nhập đúng nguyên văn rồi nhấn Enter.

5.1 Lỗi PowerShell và cài đặt (Windows)

Triệu chứng	Nguyên nhân	Cách xử lý
... cannot be loaded because running scripts is disabled	Windows chặn chạy script .ps1 mặc định	Chạy một lần (gõ Y nếu hỏi): Set-ExecutionPolicy -Scope CurrentUser RemoteSigned rồi chạy lại lệnh cũ
winget không nhận diện / winget : ...not	Thiếu công cụ cài đặt của winget	Cài App Installer từ Microsoft

Triệu chứng	Nguyên nhân	Cách xử lý
recognized	Windows	Store, hoặc tải Ollama thủ công tại ollama.com/download
Script báo “CẦN KHỞI ĐỘNG LẠI POWERSHELL”	Vừa cài Ollama, PATH chưa được nạp	Gõ exit đóng cửa sổ → mở PowerShell MỚI → cd lại thư mục → chạy lại .\0_setup\setup.ps1. Không nhấn Enter trong cửa sổ cũ.

5.2 Ollama không nhận / không chạy

Triệu chứng	Nguyên nhân	Cách xử lý
ollama: command not found / “không nhận diện ollama”	Chưa cài, hoặc đã cài nhưng PATH chưa refresh	Đóng và mở lại terminal rồi thử ollama --version. Vẫn lỗi thì cài thủ công tại ollama.com/download
Connection refused / Open WebUI không gọi được Ollama / không phản hồi	Tiến trình Ollama chưa chạy	Mở app Ollama (Windows: icon ở khay hệ thống góc phải; macOS: mở app). Kiểm tra: ollama list hoặc mở trình duyệt vào http://localhost:11434 — phải thấy chữ “Ollama is running”

Khởi động lại Ollama khi cần: Windows — chuột phải icon khay → **Quit** rồi mở lại; macOS (app) — Cmd+Q rồi mở lại; Linux — `sudo systemctl restart ollama`.

```

Windows PowerShell - ollama run qwen3:1.7b

PS C:\Users\Minh Phuc\...\bai-giang> ollama run qwen3:1.7b "Giải thích RAG trong 3 câu"

RAG (Retrieval-Augmented Generation) cho LLM đọc tài liệu của bạn trước khi trả lời: hệ thống tìm các đoạn liên quan nhất rồi đưa kèm câu hỏi vào model. Nhờ vậy câu trả lời bám tài liệu, ít bịa, và trích được nguồn để kiểm chứng.

PS C:\Users\Minh Phuc\...\bai-giang>

[100% offline] qwen3:1.7b (~1.4GB) chạy ngay trên máy · không gọi cloud

```

5.3 Docker và cổng (port) — chỉ liên quan Open WebUI

Địa chỉ tham chiếu: **Docker** → <http://localhost:3000> · **pip** → <http://localhost:8080> · **Ollama API** → <http://localhost:11434>.

Triệu chứng	Nguyên nhân	Cách xử lý
cannot connect to the Docker daemon / error	Docker Desktop chưa mở	Mở Docker Desktop , đợi trạng thái “running” (góc

Triệu chứng	Nguyên nhân	Cách xử lý
during connect ... dockerDesktopLinuxEngine		dưới chuyển xanh), rồi chạy lại docker compose up -d
Mở localhost:3000 không lên trang	Container chưa chạy	Chạy docker compose up -d, rồi docker ps phải thấy dòng open-webui
port is already allocated / cổng 3000 (hoặc 8080 nếu dùng pip) đang bận	Ứng dụng khác đang chiếm cổng	Tắt ứng dụng chiếm cổng, hoặc đổi cổng: sửa docker-compose.yml thành "3001:8080" rồi mở http://localhost:3001
Cổng 11434 đang bận	Đã có Ollama khác chạy sẵn	Thường không ảnh hưởng — dùng luôn
Docker bị chặn (máy công ty)	Không cài hoặc không bật được Docker	Cài bằng pip thay thế: pip install open-webui rồi open-webui serve → mở địa chỉ terminal in ra (thường http://localhost:8080) (lần đầu nặng ~2.5GB, ~20 phút)

5.4 Tải mô hình chậm / nghi treo

Triệu chứng	Nguyên nhân	Cách xử lý
ollama pull đứng lâu ở vài %	Mạng yếu hoặc mô hình lớn — không phải treo	qwen3:1.7b nặng ~1.4GB nên cần thời gian. Kiểm tra Internet, để quá trình chạy tiếp (không dừng giữa chừng).

5.5 Notebook / Python (Module 1)

Triệu chứng	Nguyên nhân	Cách xử lý
ModuleNotFoundError	Chưa bật môi trường ảo (venv)	Bật venv (đầu dòng phải có (.venv)): Windows . \.venv\Scripts\Activate.ps1 · macOS/Linux source .venv/bin/activate. Vẫn lỗi thì chạy lại setup.ps1/setup.sh

5.6 macOS thiếu Homebrew

Triệu chứng	Nguyên nhân	Cách xử lý
brew: command not found	Chưa cài Homebrew	Cài theo hướng dẫn tại brew.sh (dán một dòng lệnh vào Terminal), sau đó cài tiếp <code>brew install python@3.11</code> nếu cần

5.7 Open WebUI chạy nhưng kết quả sai

Triệu chứng	Nguyên nhân	Cách xử lý
Trả lời không có phần “Sources” / không bấm tài liệu	Chưa gán Knowledge, hoặc embedding chưa đổi sang nomic	(1) Trong chat gõ # chọn Knowledge base (hoặc chọn trợ lý chuyên dụng). (2) Vào Settings → Documents đặt Embedding Model = nomic-embed-text , rồi Reindex lại tài liệu
Trả lời bằng tiếng Anh / nói “Tài liệu không đề cập” dù có tài liệu	Mô hình nhỏ 1.7b đôi khi không ổn định	Thêm câu “trả lời bằng tiếng Việt” vào câu hỏi; hỏi bằng từ ngữ tự nhiên (tránh mã ngắn như “P1”); nếu vẫn lỗi thì đổi sang qwen3:4b

The image shows a web application interface for an LLM. The top section is the 'Settings' page, specifically the 'General' tab. It includes sections for 'Content Extraction Engine' (with options like PDF Extract Images (OCR), PDF Loader Mode, Bypass Embedding and Retrieval, Text Splitter, and Markdown Header Text Splitter), 'Embedding' (with fields for Embedding Model Engine, API Key, Embedding Model, Embedding Batch Size, Async Embedding Processing, and Embedding Concurrent Requests), and 'Retrieval' (with options for Full Context Mode, Hybrid Search, Reranking Batch Size, Top K, and RAG Template). The bottom section is a chat interface with a sidebar on the left showing 'Open WebUI', 'New Chat', 'Search', 'Notes', 'Workspace', 'Folders', 'Chats', and 'Today'. The main chat area shows a conversation with 'qwen3:1.7b' where the user asks: 'Quy trình xử lý sự cố ATTT gồm những bước nào? Hãy trả lời bằng tiếng Việt, liệt kê rõ từng bước.' The model's response includes a 'Retrieved 2 sources' section, a 'Thought for 3 seconds' section, and a detailed answer in Vietnamese about the steps for handling ATTT incidents, categorized by severity levels (Cấp độ 0 - Công khai, Cấp độ 3 - Tuyệt mật).

Dấu hiệu RAG thật sự chạy: dưới câu trả lời có phần trích nguồn / “Sources” trỏ đúng tên file. Không có phần đó nghĩa là đang chat thuần, chưa phải RAG.

5.8 Độ trễ ở truy vấn đầu tiên

Triệu chứng	Nguyên nhân	Cách xử lý
Gửi câu hỏi xong đợi rất lâu (15–60s) mới ra chữ, như	qwen3 bật chế độ “thinking” — sinh một đoạn lý luận dài	Thêm /no_think vào đầu câu hỏi, hoặc đưa /no_think

Triệu chứng	Nguyên nhân	Cách xử lý
treo	(thường tiếng Anh) trước khi trả lời. Câu hỏi RAG có nhiều ngữ cảnh nên đoạn này càng dài	vào System Prompt của trợ lý (mục 4.5); hoặc tắt nút “Think” (biểu tượng bộ não) cạnh ô gửi. Cách này cho trả lời gần như tức thì
Lần hỏi đầu tiên chậm hơn các lần sau	Mô hình đang được nạp vào bộ nhớ lần đầu	Bình thường — các câu sau sẽ nhanh

Phần II — Tự xây ứng dụng RAG bằng code (nâng cao)

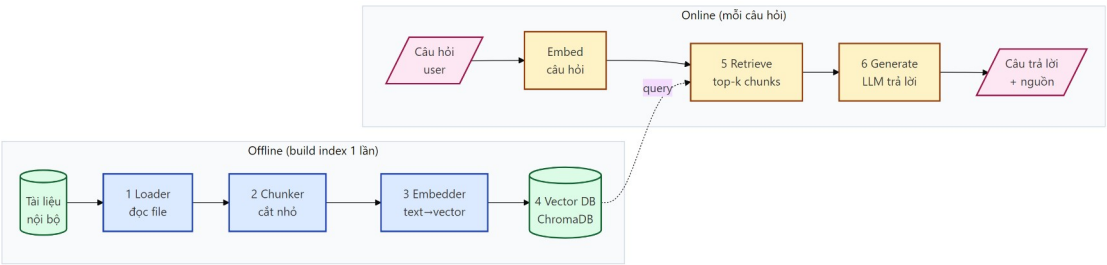
Tài liệu này dành cho việc tìm hiểu chuyên sâu và tự xây dựng RAG bằng mã nguồn, thay cho phương án sử dụng giao diện Open WebUI (không lập trình). Kết quả là một file `rag_app.py` (~130 dòng), vận hành hoàn toàn trên máy cục bộ, không yêu cầu kết nối Internet. Toàn bộ mã nguồn dưới đây đã được kiểm chứng bằng thực nghiệm và cho ra câu trả lời kèm trích nguồn.

1. Mục tiêu xây dựng

Sản phẩm là một ứng dụng RAG thực hiện chức năng: nạp tài liệu, tiếp nhận câu hỏi, và trả lời bám sát nội dung tài liệu kèm trích nguồn. Pipeline gồm 6 bước, chia thành 2 pha:

Pha	Bước	Chạy khi nào
Offline (build 1 lần)	1. Load → 2. Chunk → 3. Embed → 4. Store	--build
Online (mỗi câu hỏi)	5. Retrieve → 6. Generate	--ask "..."

Công nghệ sử dụng: Python kết hợp **Ollama** (vận hành LLM và mô hình nhúng cục bộ) và **ChromaDB** (cơ sở dữ liệu vector). Tài liệu không sử dụng LangChain nhằm trình bày rõ từng bước; LangChain về bản chất là một lớp bọc trên chính các bước này.



2. Chuẩn bị

a) **Ollama và 2 mô hình** (chi tiết cài đặt Ollama được trình bày trong **Phần I**):

```
ollama pull qwen3:1.7b          # LLM trả lời
ollama pull nomic-embed-text    # model embedding (chữ → vector)
```

b) Hai thư viện Python (nên kích hoạt venv trước):

```
pip install ollama chromadb
```

c) Tài liệu nguồn: một thư mục chứa file .md / .txt. Tài liệu này dùng sẵn 2_rag/data/ (6 file quy chế mẫu); về sau có thể thay bằng tài liệu khác.

3. Cấu hình

Tạo file rag_app.py, mở đầu bằng phần import và các hằng số cấu hình:

```
import argparse, glob, os, sys
import chromadb, ollama

# In tiếng Việt không vỡ trên console Windows (mặc định cp1252)
try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

DATA_DIR      = "2_rag/data"          # thư mục tài liệu nguồn (.md / .txt)
DB_DIR        = "chroma_db"           # nơi ChromaDB lưu vector (tự tạo)
COLLECTION    = "rag_docs"
EMBED_MODEL   = "nomic-embed-text"
LLM_MODEL     = "qwen3:1.7b"
CHUNK_SIZE    = 500                   # số ký tự mỗi đoạn
CHUNK_OVERLAP = 50                    # ký tự gối nhau giữa 2 đoạn liên kề
TOP_K         = 3                     # số đoạn lấy ra cho mỗi câu hỏi
```

Lưu ý: ba giá trị 500 / 50 / 3 tương ứng với Chunk Size, Overlap và Top-K trong lý thuyết. Việc thay đổi các giá trị này phục vụ thử nghiệm chất lượng.

4. Xây dựng từng bước — 6 bước RAG

Bước 1 — Load: đọc tài liệu

Đọc mọi file .md/.txt trong thư mục, lưu kèm tên file để phục vụ trích nguồn về sau.

```
def load_documents(data_dir=DATA_DIR):
    docs = []
    paths = sorted(glob.glob(os.path.join(data_dir, "*.md")) +
                   glob.glob(os.path.join(data_dir, "*.txt")))
    for path in paths:
        with open(path, encoding="utf-8") as f:
```



```

        docs.append({"source": os.path.basename(path), "text": f.read()})
    return docs

```

Bước 2 — Chunk: cắt thành đoạn nhỏ

LLM không xử lý được toàn bộ file một lúc; do đó tài liệu được cắt thành đoạn khoảng 500 ký tự. **Overlap 50** giúp các câu nằm vắt ngang ranh giới không bị cắt cụt nghĩa.

```

def chunk_text(text, size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
    chunks, start = [], 0
    while start < len(text):
        chunk = text[start:start + size].strip()
        if chunk:
            chunks.append(chunk)
            start += size - overlap      # dịch tới, chừa phần overlap
    return chunks

```

Bước 3 — Embed: chữ → vector (chạy local)

Mỗi đoạn được biến đổi thành một **vector** (với nomic-embed-text là 768 chiều) thông qua Ollama; dữ liệu không được gửi ra ngoài. Toàn bộ danh sách được truyền vào để nhúng theo lô nhằm tăng tốc độ.

```

def embed(texts):
    """texts: list[str] -> list[list[float]] (mỗi text một vector)."""
    resp = ollama.embed(model=EMBED_MODEL, input=texts)
    return resp["embeddings"]

```

Bước 4 — Store: lưu vào ChromaDB (build index)

Với mỗi đoạn, ChromaDB lưu đồng thời **id + vector + văn bản gốc + metadata** (tên file). Collection cũ được xóa trước để quá trình lập chỉ mục lại được thực hiện trên trạng thái sạch.

```

def build_index():
    client = chromadb.PersistentClient(path=DB_DIR)
    try:
        client.delete_collection(COLLECTION)      # build lại từ đầu cho sạch
    except Exception:
        pass
    col = client.create_collection(COLLECTION)

    ids, documents, metadatas = [], [], []
    docs = load_documents()
    for d in docs:
        for i, ch in enumerate(chunk_text(d["text"])):
            ids.append(f"{d['source']}:{i}")
            documents.append(ch)
            metadatas.append({"source": d["source"], "chunk": i})

    embeddings = embed(documents)                  # embed cả lô 1 lần

```

```
col.add(ids=ids, embeddings=embeddings, documents=documents,
        metadatas=metadatas)
print(f"[Build] {len(docs)} tài liệu -> {len(documents)} đoạn đã lưu vào
'{COLLECTION}'.")
```

Bước 5 — Retrieve: tìm top-k đoạn gần nghĩa nhất

Câu hỏi được nhúng thành vector, sau đó ChromaDB truy hồi k đoạn gần nhất (giá trị distance càng nhỏ thì mức tương đồng ngữ nghĩa càng cao).

```
def retrieve(question, k=TOP_K):
    client = chromadb.PersistentClient(path=DB_DIR)
    try:
        col = client.get_collection(COLLECTION)
    except Exception:
        raise SystemExit("Chưa có index. Chạy: python rag_app.py --build")
    q_emb = embed([question])[0]
    res = col.query(query_embeddings=[q_emb], n_results=k)
    return [
        {"text": doc, "source": meta["source"], "distance": dist}
        for doc, meta, dist in zip(res["documents"][0], res["metadatas"][0],
res["distances"][0])
    ]
```

Bước 6 — Generate: LLM trả lời dựa trên ngữ cảnh

Các đoạn truy hồi được ghép thành **ngữ cảnh**, đưa vào một **prompt** ràng buộc mô hình chỉ trả lời theo tài liệu, rồi gọi LLM. Kết quả được in kèm **trích nguồn**.

```
def generate_answer(question):
    hits = retrieve(question)
    context = "\n\n".join(f"[Nguồn: {h['source']}] \n {h['text']}" for h in
hits)
    prompt = (
        "Chỉ trả lời dựa trên TÀI LIỆU dưới đây. Nếu tài liệu không đủ thông
tin, "
        "trả lời đúng câu 'Tài liệu không đề cập'. Trả lời bằng tiếng Việt.
\n\n"
        f"=== TÀI LIỆU ===\n{context}\n\n=== CÂU HỎI ===\n{question}"
    )
    resp = ollama.chat(
        model=LLM_MODEL,
        messages=[{"role": "user", "content": prompt}],
        think=False, # qwen3: tắt 'suy nghĩ' cho output sạch
    )
    print(resp["message"]["content"].strip())
    nguon = ", ".join(sorted({h["source"] for h in hits}))
    print(f"\n--- Trích nguồn: {nguon}")
```

Quan trọng: cốt lõi của RAG nằm ở prompt này. Yêu cầu “chỉ trả lời dựa trên tài liệu, nếu thiếu thì trả lời ‘không đề cập’” là yếu tố giảm thiểu hiện tượng bịa đặt và làm cho câu trả lời có thể kiểm chứng được.

Ghép lại — CLI

```
def main():
    ap = argparse.ArgumentParser(description="Ứng dụng RAG local (Ollama + ChromaDB)")
    ap.add_argument("--build", action="store_true", help="Build index từ tài liệu")
    ap.add_argument("--ask", type=str, metavar="CÂU_HỎI", help="Đặt 1 câu hỏi")
    args = ap.parse_args()
    if args.build:
        build_index()
    elif args.ask:
        generate_answer(args.ask)
    else:
        ap.print_help()

if __name__ == "__main__":
    main()
```

Lưu ý: mã nguồn đầy đủ có sẵn tại file **rag_app.py** trong repo.

5. Chạy thử

1) Build index (chạy 1 lần; chạy lại khi đổi/thêm tài liệu)

```
python rag_app.py --build
```

2) Hỏi

```
python rag_app.py --ask "Quy trình xử lý sự cố ATTT gồm những bước nào?"
```

Kết quả thực nghiệm (rút gọn):

```
[Build] 6 tài liệu -> 25 đoạn đã lưu vào 'rag_docs'.
```

Quy trình xử lý sự cố ATTT gồm những bước sau:

1. Chuẩn bị (Preparation) – duy trì danh sách ứng cứu (CIRT), cập nhật playbook, diễn tập...
2. Phát hiện và phân tích – nhận cảnh báo SIEM/IDS, xác định ưu tiên (P1-P4), thu thập bằng chứng...
3. Cách ly (Containment) – chặn IP, disable account...

--- Trích nguồn: 03_quy_trinh_su_co.md

Phần **Trích nguồn** trả đúng file xác nhận RAG đang vận hành đúng: câu trả lời bám theo tài liệu nguồn, không bịa đặt.

6. Tùy biến theo nhu cầu

Muốn gì	Làm sao
Dùng tài liệu riêng	Đặt file .md/.txt vào thư mục DATA_DIR, sau đó chạy lại --build. (PDF: chuyển sang .txt trước.)
Đổi/thêm tài liệu	Build lại (--build); mã nguồn tự xóa index cũ rồi dựng lại.
Đoạn to/nhỏ	Chỉnh CHUNK_SIZE / CHUNK_OVERLAP, rồi --build lại.
Lấy nhiều đoạn hơn	Tăng TOP_K (3 → 5): ngữ cảnh đầy đặn hơn nhưng độ trễ cao hơn.
Trả lời chất lượng hơn	Đổi LLM_MODEL sang qwen3:4b (cần RAM 16GB).
Embedding mạnh hơn	Đổi EMBED_MODEL sang bge-m3 (1024 chiều); cần --build lại vì số chiều khác.
Đổi giọng/ngôn ngữ trả lời	Sửa prompt trong generate_answer.
Quan trọng: đổi mô hình embedding bắt buộc lập chỉ mục lại. Vector cũ và mới khác nhau về số chiều và không gian; trộn lẫn sẽ cho kết quả sai. Xóa thư mục chroma_db/ rồi chạy --build.	

7. Bổ sung giao diện web (tùy chọn)

Trường hợp cần một ô chat thay cho thao tác trên terminal, có thể dùng **Gradio** (~12 dòng).
Điều chỉnh generate_answer để **trả về chuỗi** thay vì print, sau đó:

```
pip install gradio

# rag_ui.py
import gradio as gr
from rag_app import retrieve, LLM_MODEL
import ollama

def answer(question, history):
    hits = retrieve(question)
    context = "\n\n".join(f"[Nguồn: {h['source']}] \n {h['text']}" for h in hits)
    prompt = ("Chỉ trả lời dựa trên tài liệu sau, thiếu thì nói 'Tài liệu không đề cập', ")
```

```

        f"tiếng Việt.\n\n{context}\n\nCâu hỏi: {question}")
    out = ollama.chat(model=LLM_MODEL, messages=[{"role": "user", "content":
prompt}],
                    think=False)["message"]["content"].strip()
    nguồn = ", ".join(sorted({h["source"] for h in hits}))
    return f"{out}\n\n– Trích nguồn: {nguồn}"

gr.ChatInterface(answer, title="Trợ lý RAG local").launch()

```

Chạy python rag_ui.py, sau đó mở địa chỉ Gradio được in ra (thường là http://127.0.0.1:7860). Cần chạy --build trước để có index.

Lưu ý: đây là phương án tự xây dựng giao diện. Trường hợp chỉ cần một giao diện dựng sẵn mà không lập trình, sử dụng **Open WebUI** (xem **Phần I**).

8. Lỗi thường gặp

Triệu chứng	Nguyên nhân	Cách xử lý
ModuleNotFoundError: chromadb / ollama	Chưa cài thư viện / chưa bật venv	pip install ollama chromadb; bật venv (đầu dòng có (.venv))
UnicodeEncodeError khi in tiếng Việt	Console Windows dùng cp1252	Đã có sys.stdout.reconfigure(encoding="utf-8") đầu file; hoặc đặt biến môi trường PYTHONUTF8=1
Chưa có index. Chạy: --build	Chưa build hoặc xóa nhầm chroma_db/	Chạy python rag_app.py --build trước
Connection refused / không gọi được model	Ollama chưa chạy	Mở app Ollama; kiểm tra ollama list
model "... " not found	Chưa pull model	ollama pull qwen3:1.7b và ollama pull nomic-embed-text
Đổi embedding xong kết quả sai	Index cũ dùng embedding cũ	Xóa thư mục chroma_db/ → --build lại
Trả lời lệch / “Tài liệu không đề cập” dù có	Model nhỏ 1.7b + câu hỏi cộc lốc	Hỏi bằng câu tự nhiên (tránh mã ngắn “P1”); tăng TOP_K; đổi qwen3:4b

Phụ lục — Bảng lệnh cài đặt nhanh (Ollama + Open WebUI)

Tài liệu tra cứu nhanh phục vụ việc hỗ trợ cài đặt trên **máy cá nhân mới**, ở trạng thái **chưa cài đặt phần mềm**. Mục tiêu: vận hành **LLM local** cùng **giao diện RAG**. Mô hình: qwen3:1.7b và nomic-embed-text. Quy ước: dòng có # là phần giải thích. Thực hiện từng mục **lần lượt từ trên xuống**.

A. WINDOWS — mở PowerShell (chuột phải nút Start → *Windows PowerShell*)

1) Cài Ollama (winget có sẵn trên Win 10/11). Sau khi cài, ĐÓNG & MỞ LẠI PowerShell.

```
winget install Ollama.Ollama
```

```
ollama --version
```

kiểm tra cài xong (ra số phiên bản = OK)

2) Tải model (chạy 1 lần; sau đó offline)

```
ollama pull qwen3:1.7b
```

LLM trả lời (~1.4GB)

```
ollama pull nomic-embed-text
```

embedding cho RAG (BẮT BUỘC)

3) Cài Docker Desktop (để chạy Open WebUI)

```
winget install Docker.DockerDesktop
```

→ MỞ Docker Desktop, đợi tới khi báo "running" (icon cá voi hết nhấp nháy)

4) Chạy Open WebUI (1 lệnh, không cần repo)

```
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui ghcr.io/open-webui/open-webui:main
```

→ mở trình duyệt: <http://localhost:3000>

B. macOS — mở Terminal

0) Homebrew (máy Mac mới CHƯA có – cài trước)

```
/bin/bash -c "$(curl -fsSL
```

```
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

```
eval "$(/opt/homebrew/bin/brew shellenv)" # Apple Silicon (M1-M4). Mac
```

```
Intel: dùng /usr/local/bin/brew
```

1) Cài + khởi động Ollama (hoặc tải .dmg tại ollama.com/download)

```
brew install --cask ollama
```

```
open -a Ollama
```

chạy Ollama nền

```
ollama --version
```

2) Tải model

```
ollama pull qwen3:1.7b
ollama pull nomic-embed-text
```

3) Docker Desktop + Open WebUI

```
brew install --cask docker
open -a Docker # mở, đợi báo "running"
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v
open-webui:/app/backend/data --name open-webui ghcr.io/open-webui/open-
webui:main
# → mở: http://localhost:3000
```

C. Trường hợp không sử dụng được Docker: cài bằng pip (cả 2 OS, cần Python 3.11+)

```
pip install open-webui
open-webui serve # → mở http://localhost:8080 (KHÔNG phải 8000)
```

D. Lần đầu vào Open WebUI (BẮT BUỘC cho RAG)

1. Chọn mô hình **qwen3:1.7b** ở khung chọn model phía trên.
 2. **Admin Panel → Settings → Documents**: đặt **Embedding Engine = Ollama, Model = nomic-embed-text** → Save. (Mặc định OWUI dùng embedding khác. Nếu không đặt bước này thì RAG chạy sai.)
-

E. Xử lý lỗi thường gặp

Triệu chứng	Lệnh / cách xử lý
PowerShell: ...running scripts is disabled	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned (gõ Y) rồi chạy lại
winget không nhận diện	Cài App Installer từ Microsoft Store, hoặc tải Ollama tại ollama.com/download
Gõ ollama báo không tồn tại (vừa cài)	Đóng & mở lại terminal để nạp PATH, sau đó chạy lại <code>ollama --version</code>
Connection refused / OWUI không gọi được model	Ollama chưa chạy. Mở app Ollama; kiểm tra <code>ollama list</code> , hoặc mở <code>http://localhost:11434</code> (phải thấy "Ollama is running")
Docker: cannot connect to the Docker	Mở Docker Desktop, đợi "running", rồi chạy

Triệu chứng	Lệnh / cách xử lý
daemon	lại lệnh docker
port is already allocated (cổng 3000 bạn)	Đổi cổng: thay -p 3000:8080 bằng -p 3001:8080, rồi mở http://localhost:3001
Độ trễ cao ở câu trả lời	qwen3 đang ở chế độ suy luận. Gõ /no_think trước câu hỏi, hoặc tắt nút "Think" (icon não) để giảm độ trễ đáng kể
Lần hỏi đầu chậm hơn các lần sau	Hiện tượng bình thường: mô hình đang được nạp vào RAM
Mac: brew: command not found	Cài Homebrew (mục B.0), rồi chạy lại
Gỡ và dựng lại Open WebUI	docker rm -f open-webui rồi chạy lại lệnh docker run ở mục A.4 / B.3

F. Lệnh kiểm tra / quản lý nhanh

```
ollama list      # model đã tải
ollama ps       # model đang chạy (chiếm RAM)
ollama rm <model> # xoá model, giải phóng ổ đĩa
docker ps       # container đang chạy (phải thấy 'open-webui')
docker logs open-webui # xem Log nếu OWUI lỗi
```

Yêu cầu tối thiểu: **RAM ≥ 8GB**, ổ trống ≥ 5GB. Ollama API chạy nền tại localhost:11434; Open WebUI ở localhost:3000 (Docker) hoặc localhost:8080 (pip).